

PRODUCT REQUIREMENTS DOCUMENT (PRD)

Agentic AI Digital Forensics Assistant
Bridging the Cybersecurity Skills Gap with an LLM Forensic Triage Agent

Nama Proyek	Agentic AI Digital Forensics Assistant
Versi Dokumen	v1.1
Tanggal Penyusunan	25 Juni 2026 (Update: 27 Juni 2026)
Durasi Proyek	8 Minggu (2 Bulan) - 4 Sprint x 2 Minggu
Status	Final - Tech Stack & Infrastruktur Terkonfirmasi
Leader / PM	Biel (AI System Architect & Project Manager)

1. Overview & Objective

Agentic AI Digital Forensics Assistant adalah agen berbasis Large Language Model (LLM) yang dirancang untuk menjembatani kesenjangan keterampilan keamanan siber (cybersecurity skills gap) bagi tim system programmer, dengan memandu proses forensic triage secara cepat dan akurat pada saat terjadi insiden atau krisis keamanan.

Tujuan utama sistem ini adalah memungkinkan tim yang belum memiliki keahlian forensik mendalam tetap dapat melakukan investigasi awal terhadap insiden keamanan, dengan bantuan agent yang dapat membaca log mentah, menyusun timeline serangan, dan menjelaskan indikator kompromi (IoC) dalam bahasa yang mudah dipahami.

2. Core Requirements (Mandatory)

2.1 Log and Artifact Ingestion

Sistem harus mampu mengingesti log sistem mentah (auth.log, syslog, JSON API telemetry) serta artifact disk/memory hasil akuisisi forensik.

2.2 Context-Aware Analysis

Sistem menggunakan agent framework berbasis LLM untuk menelusuri konteks historis, menyusun timeline serangan (attack timeline), dan menerjemahkan Indicator of Compromise (IoC) mentah ke dalam penjelasan bahasa natural yang mudah dipahami.

2.3 Local/Secure Deployment

Sistem mewajibkan penggunaan model open-source yang berjalan secara lokal dan mandiri, guna menjamin bahwa log sistem dan data forensik sensitif tidak pernah bocor ke layanan API pihak ketiga di luar sistem (seperti OpenAI atau Gemini Cloud).

3. Non-Mandatory Requirements (Extra Credit)

- **Autonomous Artifact Acquisition Tool** - kemampuan agent untuk terhubung via SSH ke server target, menarik log sistem yang mencurigakan, melakukan hashing (SHA-256) untuk menjaga chain of custody, lalu langsung menganalisisnya.
- **Automated Executive Report Generator** - otomatisasi pembuatan laporan insiden siap kirim ke regulator, termasuk ringkasan eksekutif, timeline dampak, dan analisis akar masalah teknis.

Kedua fitur ini bersifat opsional dan menjadi prioritas yang dapat dikorbankan terlebih dahulu jika terjadi keterbatasan waktu pengembangan, tanpa mengubah kebutuhan inti terhadap sistem.

4. Tech Stack (Final)

Tech stack berikut telah dikonfirmasi dan menjadi acuan resmi seluruh tim pengembang. Perubahan terhadap tech stack ini harus melalui persetujuan Biel selaku AI System Architect & Project Manager.

Layer	Pilihan	Catatan
Agent Framework	LangChain (Python)	Dipilih dibanding CrewAI karena use case proyek adalah single-agent dengan beberapa tools (log grepper, regex extractor, timestamp sorter), bukan multi-agent kolaboratif. Dokumentasi lebih lengkap untuk integrasi Ollama + ChromaDB.
Local LLM Runner	Ollama	Setup paling mudah untuk tim yang baru pertama kali membangun agent berbasis LLM lokal.
Model LLM	Llama-3-8B-Instruct (quantized, GGUF)	Dijalankan melalui Ollama. Pengujian model tidak dilakukan di laptop lokal, melainkan langsung di VPS begitu aktif.

Vector DB (RAG)	ChromaDB	Setup cepat, built-in persistence, integrasi native dengan LangChain. Menyimpan runbook, dokumentasi sistem, dan incident pattern historis.
Backend Framework	FastAPI	Async native - penting karena proses inference LLM bersifat lambat dan tidak boleh blocking. Auto-docs (Swagger) membantu kolaborasi dengan tim frontend.
Frontend Framework	Next.js	Routing built-in, struktur jelas untuk dashboard multi-halaman, lebih scalable untuk fitur tambahan seperti report generator.
Database Relasional	PostgreSQL	Menyimpan metadata: user, riwayat upload, informasi log. Dipilih karena lebih scalable dibanding SQLite untuk kebutuhan multi-user.
Containerization	Docker	Hanya digunakan saat deployment ke VPS, bukan saat development lokal (lihat Bagian 6 - Strategi Development).

5. Infrastruktur (VPS)

VPS final yang akan digunakan untuk deployment dan pengujian sistem:

Provider & Paket	Biznet Gio Cloud - NEO Lite LL 16.16
CPU	16 Core vCPU (dedicated)
RAM	16 GB Dedicated RAM
Storage	60 GB SSD (Storage tambahan dapat di-upgrade jika diperlukan)
Sistem Operasi	Ubuntu Server 22.04/24.04 LTS
Status Pengadaan	Menunggu persetujuan (acc) dana - target aktif pada Sprint 1-2 (Minggu 1-4)

Catatan Alokasi Resource

RAM 16 GB menjadi batasan utama yang harus dijaga, karena harus menampung beberapa service sekaligus dalam satu server yang sama. Estimasi alokasi:

- Ollama + Llama-3-8B-Instruct (quantized): ~5-6 GB RAM saat inference

- ChromaDB: <1 GB untuk skala data proyek ini
- PostgreSQL: ~200-500 MB untuk skala kecil
- FastAPI & Next.js: masing-masing beberapa ratus MB

CPU 16 vCPU melebihi kebutuhan minimum (8 vCPU), sehingga memberikan ruang lebih untuk menjalankan beberapa proses/thread inference secara bersamaan tanpa mengalami hang.

6. Strategi Development

6.1 Development Lokal (Sebelum VPS Aktif)

Karena VPS masih menunggu persetujuan dana, tim akan melakukan development kode secara native di laptop masing-masing tanpa Docker, dengan ketentuan sebagai berikut:

- Seluruh logic non-LLM (parser log, agent tools/regex extractor/timestamp sorter, API endpoint, komponen frontend) dikembangkan dan dites secara native di lokal.
- Model LLM (Llama-3-8B-Instruct) TIDAK dites di laptop lokal. Pengujian model, inference, dan integrasi RAG penuh baru dilakukan setelah VPS aktif.
- Tidak dibuat lapisan mocking untuk LLM - begitu VPS aktif, tim langsung terintegrasi dengan model asli.
- Docker baru digunakan saat deployment ke VPS, bukan selama fase development lokal.

6.2 Trigger Keputusan Jika VPS Tertunda

Target VPS aktif adalah pada Sprint 1-2 (Minggu 1-4), namun status persetujuan dana belum final. Untuk mengantisipasi keterlambatan tanpa mengganggu jadwal kerja tim, disepakati trigger keputusan berikut:

*Jika di akhir Minggu 4 (akhir Sprint 2) VPS masih belum aktif, **Biel selaku PM akan melakukan eskalasi kepada dosen pembimbing terkait status dana, dan mempertimbangkan opsi darurat seperti meminjam resource komputasi kampus atau menyewa VPS jangka pendek sementara untuk kebutuhan testing.***

Hal ini penting karena Sprint 3 (Context-Aware Analysis + Local Deployment) bergantung penuh pada ketersediaan VPS untuk validasi akhir agent dan model LLM.

7. Struktur Tim & Tanggung Jawab

Nama	Role	Fokus Utama
Biel	AI System Architect & Project Manager (Leader)	Arsitektur sistem, pemilihan & konfigurasi LLM, sprint planning, code review, audit keamanan, komunikasi ke dosen pembimbing, deployment final.

Monik	UI/UX Designer & Frontend Developer	Dashboard, halaman upload/ingestion, halaman hasil analisis IoC.
Eca	UI/UX Designer & Frontend Developer	Timeline visualisasi attack, halaman report (jika Extra Credit dikerjakan).
Nopal	AI & Backend Developer	Log/artifact ingestion pipeline, setup Vector DB (ChromaDB) & RAG.
Rafa	AI & Backend Developer	Agent orchestration (LangChain), context-aware analysis (timeline builder, IoC translator).

8. Timeline Sprint (8 Minggu / 4 Sprint)

Sprint	Fokus	Sprint Goal
Sprint 1 (Mgg 1-2)	Setup & Fondasi	Arsitektur fix, environment siap, desain awal selesai.
Sprint 2 (Mgg 3-4)	Core 1: Ingestion + FE Mulai	Ingestion pipeline jalan, frontend mulai dibangun paralel.
Sprint 3 (Mgg 5-6)	Core 2 & 3: Analysis + Local Deployment	Agent dapat menganalisis & menjelaskan IoC; seluruh proses berjalan lokal di VPS. Sprint paling kompleks - membutuhkan VPS sudah aktif.
Sprint 4 (Mgg 7-8)	Integrasi & Finalisasi	Sistem stabil, terintegrasi penuh, siap didemokan ke dosen pembimbing.

Detail checklist task per sprint dan per anggota tim didokumentasikan secara terpisah dalam board manajemen proyek (Jira), berdasarkan pembagian tanggung jawab pada Bagian 7.

10. API Contract — Update (v1.1)

Bagian ini mendokumentasikan hasil pengecekan konsistensi API contract terhadap kebutuhan Frontend dan Backend yang dilakukan pada Wrap-up Sprint 1. Beberapa penyesuaian skema diputuskan agar selaras dengan mockup yang telah dibuat, sekaligus tetap dipetakan ulang terhadap Core Requirements pada Bagian 2.

10.1 Endpoint /upload — Tidak Berubah

Mockup frontend menampilkan dua kategori visual saat upload (System Log vs Disk/Memory Artifact), termasuk istilah teknis seperti “Volatility/SleuthKit” untuk proses ekstraksi memory forensic.

Keputusan: endpoint /upload tetap berupa satu endpoint generik yang menerima satu file (sesuai desain awal Bagian 1), tanpa parameter kategori tambahan. Mockup disederhanakan — referensi ke “Volatility/SleuthKit” dan status ekstraksi otomatis (misal “EXTRACTING NETWORK SOCKETS”) dihapus dari tampilan, karena berada di luar scope tech stack yang telah dikonfirmasi pada Bagian 4. Tab System Log vs Disk/Memory Artifact tetap dipertahankan sebagai pengelompokan visual/UX saja, bukan logic backend yang berbeda.

10.2 Endpoint /analyze — Skema Diperluas

Tabel “Triage Insiden Terbaru” pada mockup Dashboard menampilkan field severity, status investigasi, IP/IoC, dan deskripsi naratif per insiden — field-field ini sebelumnya tidak tercakup dalam skema /analyze versi awal (Bagian 1), yang hanya mengembalikan timeline, ioc_explanation, dan severity secara umum (bukan per entry).

Keputusan: skema response /analyze diperluas menjadi daftar insiden (AnalyzedIncident), masing-masing dengan field severity, status, ioc_ip, dan narrative_description, agar Dashboard dapat menampilkan data per-insiden secara langsung.

Skema AnalyzedIncident

Field	Sumber	Status terhadap Requirement
timestamp, source, event_type	Hasil parsing (ParsedLogEntry)	Wajib — bagian dari Log and Artifact Ingestion (2.1)
severity	Hasil analisis agent	Wajib — mendukung forensic triage (5.5.1), prioritas insiden saat krisis
ioc_ip	Agent tool regex_extract_ioc	Wajib — persis “translate raw indicators of compromise (IoCs)” (Core Requirement 2.2)
narrative_description	Output LLM (Llama-3-8B-Instruct)	Wajib — persis “plain English explanations” (Core Requirement 2.2)
status (New/Analyzing/Done)	Keputusan tim (UX)	Pilihan, bukan requirement wajib. Tidak disebutkan dalam dokumen requirement asli. Dipertahankan untuk kebutuhan UX Dashboard, namun menjadi kandidat pertama yang dapat disederhanakan (misal: dianggap selalu “Done”) jika waktu Sprint 3 terbatas, tanpa mengorbankan Core Requirement.

Implikasi teknis: klasifikasi event_type yang lebih naratif (misal “SSH Brute Force Attempt”) membutuhkan logic agregasi pola pada beberapa log entry (bukan klasifikasi satu baris log seperti parser dasar pada Bagian 3), dan field severity/status memerlukan PostgreSQL sebagai source of truth agar konsisten antar request.

11. Risiko & Mitigasi

Risiko	Dampak	Mitigasi
VPS belum aktif saat Sprint 3 dimulai	Nopal & Rafa tidak dapat memvalidasi agent & model LLM secara penuh	Trigger eskalasi ke dosen pembimbing di akhir Minggu 4 (lihat Bagian 6.2)
RAM 16 GB tidak cukup menampung seluruh service bersamaan	Inference lambat, service crash, atau out-of-memory	Monitoring alokasi resource ketat (lihat Bagian 5), prioritaskan Ollama saat inference berjalan
Waktu pengembangan terbatas (2 bulan)	Fitur Extra Credit tidak terselesaikan	Extra Credit (Autonomous Acquisition, Report Generator) dikorbankan lebih dulu; fokus pada 3 Core Requirements
Sprint 3 terlalu padat (2 Core Requirement sekaligus)	Progress molor, bug ditemukan terlambat	Daily check-in oleh Biel khusus selama Sprint 3; realokasi tugas FE-BE jika diperlukan